COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS

SEMESTER 2 2025/2026

SOFTWARE CONSTRUCTION & METHODS (CSEB5223)

LAB 9 & LAB 10 ASSIGNMENT (FINAL REPORT)

SEMESTER 2 2025/2026

PROGRAMME : BACHELOR OF SOFTWARE ENGINEERING (HONS.)
GROUP ASSIGNMENT : GROUP 6
PROJECT TITLE : STUDENT LEARNING MANAGEMENT SYSTEM (SLMS)
LECTURER NAME : TS. DR. LOO YIM LING
GROUP MEMBERS :

No.	Full Name	Student ID
1	ARIFAH NUR IMAN BINTI MOHD FADZILAH	BSW01085214
2	REYHANA MARSYA BINTI AHMAD SAIFULLIZAN	BSW01085069
3	DESHINI A/P SELVAKUMAR	SW01083593
4	PATVIR SINGH KHOSA	BSW01085393

GITHUB REPOSITORY URL: https://github.com/Gwigle/SLMS-CSEB5223-Group6.git

Table of Contents

1.0 Introduction 4

2.0 Use Case Design 7

2.1 Use Case 7

2.2 Use Case Diagram 8

3.0 Class Design 9

3.1 Class Relationships 11

3.2 UML Class Diagram 12

4.0 Documentation Formatting 13

5.0 Coding File Formatting 13

6.0 Naming Conventions – Dictionary 14

6.1 Naming Conventions 14

6.2 Naming Dictionary 15

7.0 Function and Methods Modularity 18

7.1 Principles of Modularity Applied in SLMS 18

7.2 Method-Level Modularity 18

7.3 Reusability of Methods 19

7.4 Low Coupling and High Cohesion 20

7.5 Summary of Modularity Benefits in SLMS 20

8.0 Continuous Integration Rules 21

8.1 Repository Structure 22

9.0 Course Profile 23

9.1 Class Design 23

9.2 Course Input Implementation 26

9.3 Search Function Development 32

9.4 Edit Function Development 34

9.5 Delete Function Development 38

9.6 Data Display Function Development 43

10.0 Student Profile 45

10.1 Class Design 45

10.2 Course Input Implementation 46

10.3 Search Function Development 51

10.4 Edit Function Development 53
10.5 Delete Function Development 55
10.6 Data Display Function Development 58
11.0 Course – Student Relationship and middleware API 60
11.1 Data Structure Design 60
11.2 Relationship Management Functions 63
11.3 Error Handling and Edge Cases 66
11.4 API Implementation 74
12.0 Analysis of software construction best practices and challenges 79
12.1 Lab 3 & Lab 4 79
12.2 Lab 5 & Lab 6 80
12.3 Lab 7 & Lab 8 81
13.0 System Testing 82
14.0 Team Contributions 91
15.0 References 92

1.0 Introduction

The Student Learning Management System (SLMS) is a console-based Java application designed to allow administrators to efficiently manage students, courses and their relationships in a centralized system. The system implements a menu-driven interface, enabling easy input, search, editing, deletion and display of student and course profiles.

In Lab 2, the focus was on drafting the system plan, including use case design, class design, documentation, coding standards, naming conventions, modular functions and continuous integration practices. This established a strong foundation for structured software construction.

In Lab 3, we implemented course input, search and display functionalities in Course Profile. Admin can add courses with attributes such as course code, name, credit hours, summary, MS Teams link and type (core or elective). Input validation ensures data integrity, while the displayCourse() method provides a clear, formatted console output.

In Lab 4, the system was extended to support editing, deletion and viewing of all course profiles, with robust error handling for invalid inputs and prevention of duplicate entries. The system now maintains an array-based storage of courses, with a relationship matrix to track course assignments.

In Lab 5, the Student Profile module was introduced. A Student class was created with attributes such as name, student ID, email and phone number, along with getter and setter methods. The system allows input of student data, stores multiple records in an array and prevents out-of-bound errors. A linear search function was implemented to find students by ID, and editing functionality allows modification of student details (except student ID) with proper validation and display.

In Lab 6, the Student Profile module was completed with delete and view functionalities. Admin can delete student records with confirmation, and the system updates and displays all records after deletion. A “View All” feature was added to display all student profiles in a clear format. Finally, both Course and Student modules were fully integrated into a single system, ensuring smooth functionality, consistent data handling and improved user interaction.

In Lab 7, the system was enhanced to support Course-Student relationships. A many-to-many relationship was implemented using a two-dimensional boolean array, where each row represents a student and each column represents a course. Functions such as assigning students to courses, listing courses of a student and listing students of a course were developed. Additional validation checks were introduced to handle edge cases such as non-existent students or courses, duplicate assignments and empty relationships, ensuring system robustness and data consistency.

In Lab 8, a simple middleware API was introduced to improve system usability. A SearchCacheManager class was implemented to store recent inputs and search results, enabling auto suggestion features for course codes and student IDs. This reduces repetitive input and simulates real-world API behavior. The system was further refactored into separate manager classes (CourseManager, StudentManager and RelationshipManager) to improve modularity, maintainability and separation of concerns. All components were then integrated into the main system, providing a more structured and user-friendly application.

In Lab 9, testing was carried out to ensure system correctness and reliability. Unit tests verified individual functions, while integration testing ensured all modules worked together smoothly. Additional error handling and defensive programming were implemented to manage invalid inputs and edge cases. Our report included screenshots of system behavior and analysis of best practices, with all updates reflected in the GitHub repository.

In Lab 10, the project was finalized through documentation and code refinement. Design decisions, module descriptions and API details were compiled, while code readability and consistency were improved through proper formatting, naming conventions and comments. The GitHub repository was updated to present a complete and well-structured system.

Overall, the SLMS project successfully demonstrates the application of Object-Oriented Programming principles, modular design and systematic development practices. From initial planning to implementation, testing and documentation, the system evolved into a robust and maintainable application. The integration of validation, error handling and structured testing ensures reliability, while clear documentation supports future scalability and enhancement. Furthermore, GitHub version control supports collaborative development and continuous integration.

2.0 Use Case Design

The Use Case Design describes how users interact with the Student Learning Management System (SLMS). It identifies the system of actor and the different actions that can be performed within the system. In SLMS, the main actor is the admin, who is responsible for managing courses and student information.

The use case diagram in Figure 1 illustrates the functionalities available in the system such as adding, searching, editing and deleting courses and students, as well as assigning students to courses. These interactions help to define the system requirements clearly and provide a visual understanding of how the system operates.

2.1 Use Case

Actor	**Use Case**	**Description**
Admin	Add course	Admin inputs course information and adds it to the system.
Admin	Search course	Admin searches for a course by course code.
Admin	Edit course	Admin edits any course attributes except course code.
Admin	Delete course	Admin deletes a course after confirmation; removes relationships.
Admin	View all courses	Admin views all courses with complete details.
Admin	Add student	Admin inputs student information and adds it to the system.
Admin	Search student	Admin searches for a student by student ID.
Admin	Edit student	Admin edits student attributes except student ID.
Admin	Delete student	Admin deletes a student after confirmation; removes relationships.
Admin	Assign Student to Course	Admin assigns a student to a course.
Admin	List Courses of Student	Admin views all courses a student is enrolled in.
Admin	List Students of Course	Admin views all students enrolled in a course.

2.2 Use Case Diagram

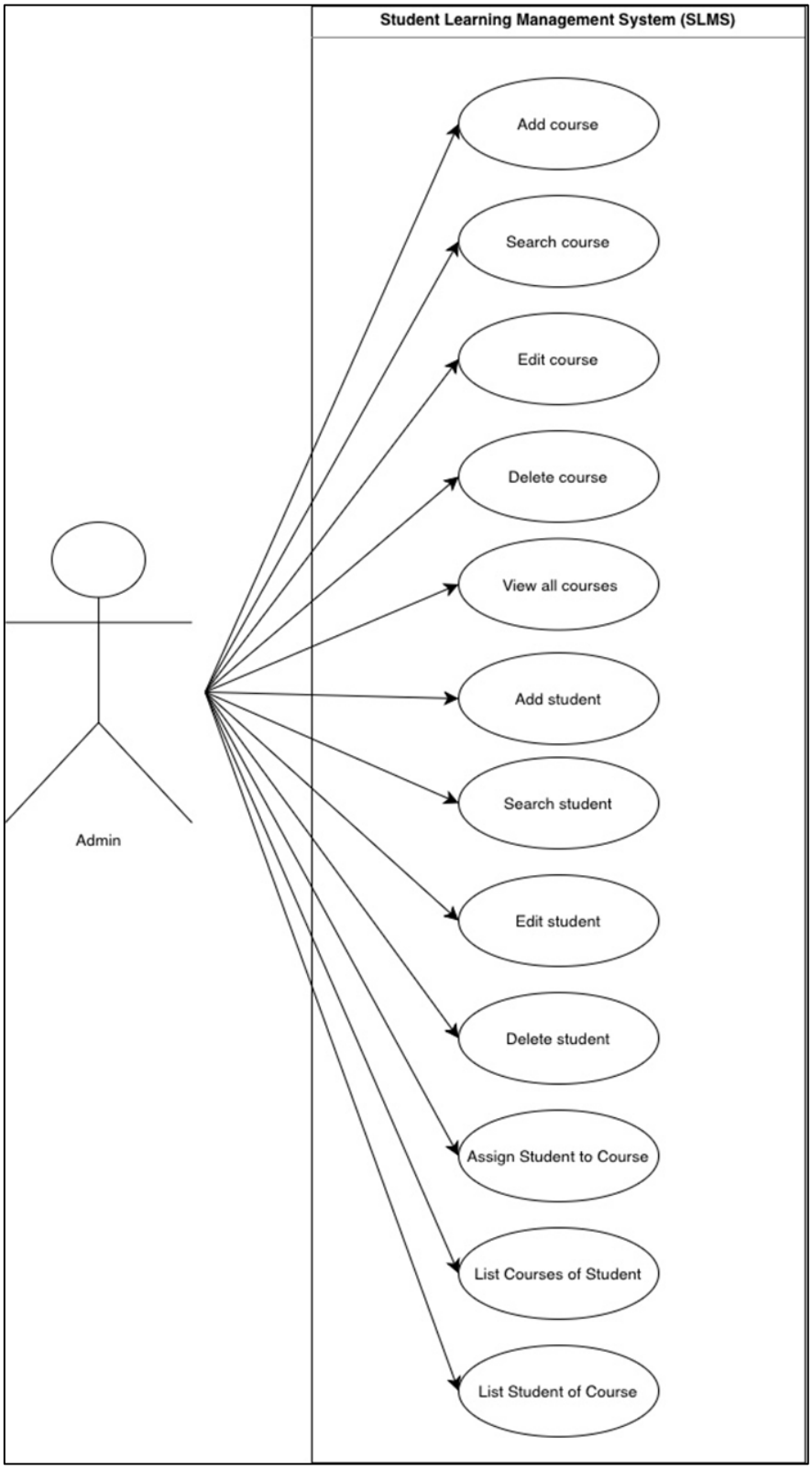

Figure 1

3.0 Class Design

The Class Design section provides a detailed blueprint of the system's structure by defining the classes, their attributes, methods, and the relationships between them. In the Student Learning Management System (SLMS), the class design follows object-oriented programming principles to ensure modularity, reusability and maintainability.

Class	**Attribute**	**Method**
Course	courseCode: String courseName: String creditHour: int summary: String teamsLink: String courseType: String	getCourseCode( ) getCourseName( ) setCourseName( ) getCreditHour( ) setCreditHour( ) getSummary( ) setSummary( ) getTeamsLink( ) setTeamsLink( ) getCourseType( ) setCourseType( ) displayCourse( )
Student	studentID: String studentName: String email: String phone: String	getStudentID( ) getStudentName( ) setStudentName( ) getEmail( ) setEmail( ) getPhone( ) setPhone( ) displayStudent( )
CourseManager	courses: Course[] courseCount: int sc: Scanner	addCourse() findCourseIndex() searchCourse() getCourseByCode() editCourse()

		deleteCourse() viewAllCourses() getCourses() getCourseCount()
StudentManager	courses: Course[] courseCount: int sc: Scanner	addStudent() findStudentIndex() searchStudent() getStudentById() editStudent() deleteStudent() viewAllStudents() getStudents() getStudentCount()
RelationshipManager	relationship : boolean[][] courseManager : CourseManager studentManager : StudentManager sc : Scanner	assignStudentToCourse() addCourse() addStudent() findCourse() listCourses() findStudent() listStudents() removeCourseRelationship() removeStudentRelationship()
SearchCacheManager	recentCourseSearches: String[] recentStudentSearches: String[] courseSearchCount:int studentSearchCount: int	cacheCourseSearch() cacheStudentSearch() suggestCourses() suggestStudents()
Main	None	main(String[ ] args)

3.1 Class Relationships

Association Relationships

As shown in Figure 2, the SLMS uses several association relationships between classes. Association represents a structural relationship where one class is aware of or uses another class. These relationships show how different parts of the system interact while remaining modular.

Class A	Relationship	Class B	Multiplicity	Description
RelationshipManager	uses	CourseManager StudentManager SearchCacheManager	1 : 1	The RelationshipManager depends on CourseManager, SearchCacheManager to access course data, course indexes, student data and student indexes.
SearchCacheManager	caches input for	CourseManager StudentManager	1 : 0..*	The cache manager stores user search values before or during search execution.
CourseManager	manages	Course	1 : 0..*	One CourseManager can manage zero or many Course objects stored in the courses array.
StudentManager	manages	Student	1 : 0..*	One StudentManager can manage zero or many Student objects stored in the students array.

3.2 UML Class Diagram

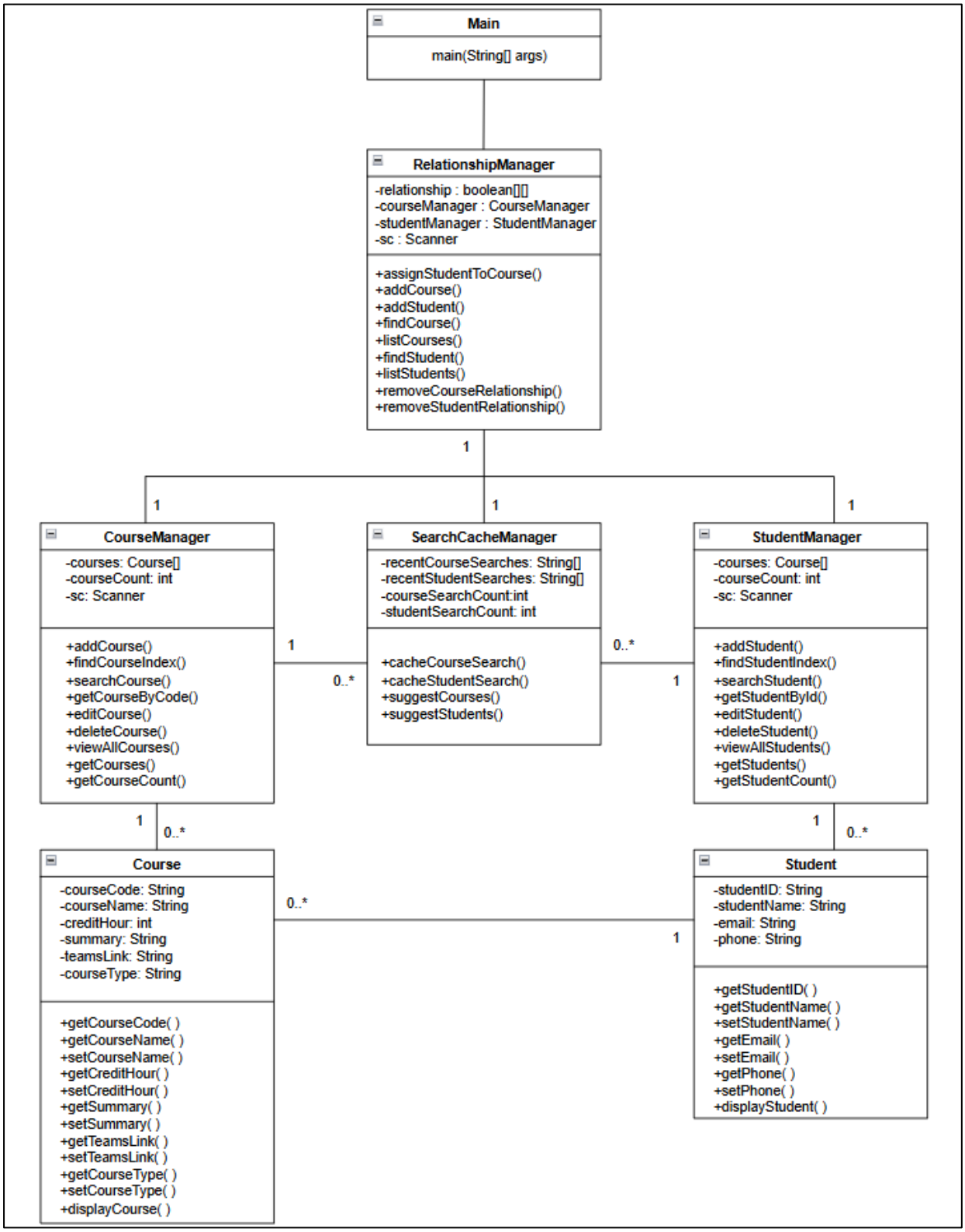

Figure 2

4.0 Documentation Formatting

The following documentation formatting standards are used in preparing our SLMS report:

- Font: Times New Roman, 12 pt
- Headings: Bold, Times New Roman, 12 pt
- Paragraph Alignment: Justified
- Line Spacing: 1.5
- Section Numbering: Hierarchical numbering is used for readability and structure
- Figures: Clearly labeled figure number and draw picture border
- Tables: Clearly displayed and referenced within the report

5.0 Coding File Formatting

To maintain consistency, readability and standardization, the SLMS project follows the coding file formatting guidelines below:

- Programming Language: Java
- File Naming: Each class is stored in its own .java file
- Class-to-File Matching: File names match class names exactly (Course.java)
- Indentation: 4 spaces are used for all code blocks
- Brackets Style: Opening braces { are placed on the same line as class and method declarations
- Comments: Single-line comments use //, multi-line comments use /* ... */
- File Header: Each file begins with a header comment containing file name, authors, description
- Line Length: Code lines are kept reasonably short to avoid excessive horizontal scrolling
- Naming Consistency: Variables, methods and classes follow pre-defined naming conventions

6.0 Naming Conventions – Dictionary

Naming conventions are a pre-defined set of rules and/or guidelines used to give elements, variables, class, and other types of code readable and consistent names/identifiers to ensure readability, consistency and maintainability.

6.1 Naming Conventions

Naming conventions help developers to write readable and consistent code by applying the set of rules and/or guidelines to denote identifiers.

Element	**Convention**	**Example**
Class	PascalCase	Course,Student,CourseManager,StudentManager, RelationshipManager,SearchCacheManager, Main
Method	camelCase	addCourse(), searchCourse(), editCourse(), deleteCourse(), viewAllCourses()
Variable	camelCase	courseCode, courseName, creditHour, summary, teamsLink, courseType
Constant	UPPER_CASE	MAX_COURSES, MAX_STUDENTS
File Name	Same as class name	Course.java, RelationshipManager.java
Boolean Variable	Starts with is/has	isEnrolled, hasSubmitted

6.2 Naming Dictionary

A naming dictionary acts as a glossary to define naming conventions and identify what type of code it is.

Name	**Type**	**Description**
Course	Class	Stores course information
Student	Class	Stores student information
CourseManager	Class	Handles all course-related operations
StudentManager	Class	Handles all student-related operations
RelationshipManager	Class	Handles student-course assignments and relationships
SearchCacheManager	Class	Handles cached searches and auto-suggestions
Main	Class	Main entry point of the system
getCourseCode()	Method	Returns the course code
getCourseName()	Method	Returns the course name
setCourseName()	Method	Updates the course name
getCreditHour()	Method	Returns the course credit hour
setCreditHour()	Method	Updates the course credit hour
getSummary()	Method	Returns the course summary
setSummary()	Method	Updates the course summary
getTeamsLink()	Method	Returns the Teams link
setTeamsLink()	Method	Updates the Teams link
getCourseType()	Method	Returns the course type
setCourseType()	Method	Updates the course type
displayCourse()	Method	Displays course information

getStudentID()	Method	Returns the student ID
getStudentName()	Method	Returns the student name
setStudentName()	Method	Updates the student name
getEmail()	Method	Returns the student email
setEmail()	Method	Updates the student email
getPhone()	Method	Returns the student phone number
setPhone()	Method	Updates the student phone number
displayStudent()	Method	Displays student information
addCourse()	Method	Adds a new course or creates a course relationship
searchCourse()	Method	Searches for a course
editCourse()	Method	Edits course details
deleteCourse()	Method	Deletes a course
viewAllCourses()	Method	Displays all courses
findCourseIndex()	Method	Returns the index of a course in the array
getCourseByCode()	Method	Returns a course object using its code
addStudent()	Method	Adds a new student or creates a student relationship
searchStudent()	Method	Searches for a student
editStudent()	Method	Edits student details
deleteStudent()	Method	Deletes a student
viewAllStudents()	Method	Displays all students
findStudentIndex()	Method	Returns the index of a student in the array
getStudentById()	Method	Returns a student object using its ID
assignStudentToCourse()	Method	Assigns a student to a course

listCourses()	Method	Lists all courses of a student
listStudents()	Method	Lists all students in a course
findCourse()	Method	Finds a student's course
findStudent()	Method	Finds student in a course
removeCourseRelationship()	Method	Removes relationships after course deletion
removeStudentRelationship()	Method	Removes relationships after student deletion
cacheCourseSearch()	Method	Stores recent course searches
cacheStudentSearch()	Method	Stores recent student searches
suggestCourses()	Method	Suggests cached course codes
suggestStudents()	Method	Suggests cached student IDs

7.0 Function and Methods Modularity

Modularity in software construction and methods refers to dividing a system into separate, independent modules where each module contains everything needed to execute a specific functionality. In the Student Learning Management System (SLMS), we apply modularity principles to ensure our methods are reusable and maintainable.

7.1 Principles of Modularity Applied in SLMS

1. **Single Responsibility Principle (SRP)**

   Each class and method performs only one clear responsibility. Example:

   - getCourseCode() in Course class only handles in getting course codes.
   - getStudentID() in Student class only handles getting Student IDs.

2. **Separation of Concerns**

   Each class handles a specific part of the system:

   - User-related operations – StudentManager, Student
   - Course management – Course

3. **Encapsulation**

   All attributes are declared as private and accessed using getter and setter methods.

7.2 Method-Level Modularity

Each method is:

- short and concise
- named using CapitalCase
- programmed to follow one specific instruction
- can be reusable

7.3 Reusability of Methods

Reusability ensures that methods can be used multiple times throughout the system without rewriting the same code.

- Reduces code duplication in the system.
- Improves code maintainability and readability.
- Allows methods to be reused in multiple functions.

Examples in SLMS system

1. displayCourse()

- Used to display course information.
- Reused in features like viewAllCourses() and searchCourse().

2. displayStudent()

- Displays student information.
- It can be reused when viewing or searching for student records.

3. Getter methods such as:

- getCourseCode()
- getCourseName()
- getStudentID()

These methods allow other classes to access object data safely without directly modifying it. Using reusable methods makes the SLMS system easier to maintain, update and expand in future development.

7.4 Low Coupling and High Cohesion

Low coupling and high cohesion help create a well-structured and maintainable software system.

High Cohesion

- Each class focuses on one specific responsibility.
- The Course class manages course information.
- The Student class manages student information.
- The StudentManager class manages system operations such as adding, searching, editing and deleting data students.

Low Coupling

- Classes do not depend heavily on each other.
- The Course and Student classes only store data and methods related to themselves.
- The RelationshipManager class controls interactions between students and courses.

Benefits of applying these principles

- Improve system organization
- Makes debugging easier
- Supports future system expansion

7.5 Summary of Modularity Benefits in SLMS

A. Maintainability - Small, focused methods are easier to update and debug
B. Reusability - Utility methods and service classes prevent code duplication
C. Readability - Method names clearly indicate what each function does
D. Scalability - New features can be added by creating new methods without breaking existing ones
E. Collaboration - Team members can work on different methods simultaneously with minimal conflicts

8.0 Continuous Integration Rules

Continuous Integration (CI) is a software development practice that ensures code changes from all team members are regularly integrated, tested and merged into the main branch. In the SLMS project, CI helps maintain a stable, working codebase, detect issues early and streamline collaboration among students, lecturers and administrators.

Branching and Pull Requests

- The main branch is protected, and no direct commits are allowed.
- Developers work on feature or bugfix branches, named clearly to reflect their purpose.
- All code changes are submitted via Pull Requests (PRs), reviewed by at least one team member.
- The team leader approves and merges PRs, ensuring quality control.

Commit Message Standards

- Commit messages follow the format: [Module] Short Description
- Guidelines: present tense, concise, descriptive and avoid vague messages.
- Examples:
    - [Course] Add search function
    - [Student] Add profile editing function

Testing and Code Validation

- Code must compile and pass all local tests before submission.
- Unit tests ensure new features do not break existing modules.
- Automated Pull Request checks verify runtime stability and functionality.

8.1 Repository Structure

- Organized folders support modularity, maintainability and collaboration.
- Our documentation is kept up to date with each code addition.

```
SLMS
├── docs
│       ├── SLMS-CSEB5223-Group6.docx
│       ├── SLMS-CSEB5223-Group6-(Version2).pdf
│       ├── SLMS-CSEB5223-Group6-(Version3).pdf
│       ├── SLMS-CSEB5223-Group6-(Version4).pdf
│       └──README.md
├── src
│       ├── Course.java
│       ├── Student.java
│       ├── CourseManager.java
│       ├── StudentManager.java
│       ├── RelationshipManager.java
│       ├──SearchCacheManager.java
│       ├── Main.java
│       └── README.md
└──README.md
```

Collaboration Practices

- Team members must pull before pushing to prevent conflicts.
- Clear communication is required for changes via GitHub issues or project boards.
- Code reviews help share knowledge and enforce best practices

9.0 Course Profile

The Course Profile module of SLMS is responsible for managing course data, including input, search, edit, delete and display functionalities.

9.1 Class Design

The Course class encapsulates all relevant attributes for a course and provides getter, setter and display methods as shown in Figure 3 and Figure 4. It also contains a comment block of the file name which is Course.java, our names and a brief description of the Course class.

Course.java

Source History

```java
/*
File:
Course.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Represents a course in the SLMS system with attributes, getters, setters, and display functionality.
*/

public class Course {

    // Attributes to store course information
    private String courseCode; // Unique course code
    private String courseName; // Name of the course
    private int creditHour;    // Number of credit hours
    private String summary;    // Brief description of the course
    private String teamsLink;  // Microsoft Teams link
    private String courseType; // Type: core, elective, university

    // Constructor: Initializes all course attributes when a Course object is created
    public Course(String courseCode, String courseName, int creditHour,
                  String summary, String teamsLink, String courseType) {
        this.courseCode = courseCode;
        this.courseName = courseName;
        this.creditHour = creditHour;
        this.summary = summary;
        this.teamsLink = teamsLink;
        this.courseType = courseType;
    }

    // ------------------------- GETTERS -------------------------
    public String getCourseCode() { return courseCode; }
    public String getCourseName() { return courseName; }
    public int getCreditHour() { return creditHour; }
    public String getSummary() { return summary; }
    public String getTeamsLink() { return teamsLink; }
    public String getCourseType() { return courseType; }

```

Figure 3

```java

    // ------------------------- SETTERS -------------------------
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public void setCreditHour(int creditHour) { this.creditHour = creditHour; }
    public void setSummary(String summary) { this.summary = summary; }
    public void setTeamsLink(String teamsLink) { this.teamsLink = teamsLink; }
    public void setCourseType(String courseType) { this.courseType = courseType; }

    // Display course information in a readable format
    public void displayCourse() {
        System.out.println("==================================");
        System.out.printf("Course Code : %s%n", courseCode);
        System.out.printf("Course Name : %s%n", courseName);
        System.out.printf("Credit Hour : %d%n", creditHour);
        System.out.printf("Summary     : %s%n", summary);
        System.out.printf("Teams Link  : %s%n", teamsLink);
        System.out.printf("Type        : %s%n", courseType);
        System.out.println("==================================");
    }
}
```

Figure 4

Attributes

These private variables store the specific data for each course. The access modifier "private" is a practice called encapsulation, which protects the data from being changed directly from outside the class.

Attribute	Type	Description	Example Input
courseCode	String	Unique identifier of the course.	"CSEB5223"
courseName	String	Full name of the course.	"Software Construstion & Method"
creditHour	int	Number of credit hours.	3
summary	String	Brief description of the course.	"Practice coding and build SLMS project."
teamsLink	String	Microsoft Teams link for course resources.	"https://teams.microsoft.com/l/course/CSEB5223"
courseType	String	Type of course: core, elective or university.	"core"

Getter and Setter Methods

Since the variables are private, other classes cannot use them. Hence, getter methods allow other parts of the program to access course details like name, credit hours, summary, Teams link and type. Setter methods update these details after creation, except for courseCode, which is immutable.

Course Display

The displayCourse() method neatly prints all course information using System.out.println() and System.out.printf(), with formatting symbols (%s for String, %d for int) and separators for clarity.

Course Array Initialization

As shown in Figure 5, the courses array is declared in the CourseManager class to store all course records in the system. The array is initialized with a fixed size of 100, meaning the system can store up to 100 courses only.

```java
/*
File:
CourseManager.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Handles all course-related operations such as add, search, edit, delete and view.
*/

import java.util.Scanner;

public class CourseManager {

    private Scanner sc;
    private Course[] courses = new Course[100]; // Fixed-size storage
    private int courseCount = 0; // Tracks number of stored courses

    // Constructor:  Initializes scanner and loads sample data
    public CourseManager(Scanner sc) {
        this.sc = sc;
        loadSampleCourses();
    }
```

Figure 5

Sample Data Initialization

Sample data in Figure 6 is initialized inside the constructor of CourseManager class. This allows the system to already contain course records when it starts

```java

    // Load sample courses
    private void loadSampleCourses() {
        courses[0] = new Course("CSEB5223", "Software Construction & Methods", 3,
                "Practice coding and build SLMS project.", "https://teams.microsoft.com/l/course/CSEB5223", "core");
        courses[1] = new Course("CSEB4113", "Software Quality", 3,
                "Learn software quality and testing.", "https://teams.microsoft.com/l/course/CSEB4113", "core");
        courses[2] = new Course("CCSB5113", "Software Security", 3,
                "Fundamentals of software security.", "https://teams.microsoft.com/l/course/CCSB5113", "elective");
        courseCount = 3;
    }
```

Figure 6

9.2 Course Input Implementation

The Course Input Implementation allows the admin to add new course information into the Student Learning Management System (SLMS) through console input.

Based on the lab requirements, the system must allow the admin to enter course details and store them in the system. In our implementation, the submit functionality is handled by the addCourse() method in the CourseManager class.

When the admin selects the Add Course option from the menu, the system prompts the user to enter the required course information such as:

- Course Code
- Course Name
- Credit Hour
- Course Summary
- Microsoft Teams Link
- Course Type (Core or Elective)

The system then checks whether the course code already exists to prevent duplicate entries. If the course code is unique, a new Course object is created and stored in the courses array. The courseCount variable is then increased to keep track of the number of courses stored in the system. This system stores multiple courses using an array and it also prevent array out-of-bound errors.

This implementation ensures that course information is properly stored and managed while maintaining data integrity in the SLMS.

Scenario 1: Admin adds a course when array is full

If the courses array has reached 100 entries, the system checks courseCount >= courses.length before reading input from CourseManager class as shown in Figure 8. It prints Course list full! and stops the add operation as shown in Figure 7, preventing an ArrayIndexOutOfBoundsException. No changes are made, and the system remains stable as demonstrated in Figure 7.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course list full!
```

Figure 7

```
39
40      // ------------------------- COURSE FUNCTIONS -------------------------
41
42      // Add a new course
43      public void addCourse() {
44          if(courseCount >= courses.length) {
45              System.out.println("Course list full!"); // Prevent overflow of array
46              return;
47          }
```

Figure 8

Scenario 2: Admin attempts to add a duplicate course

According to Figure 11, if the admin enters a course code that already exists, findCourseIndex() finds the duplicate and returns its index. The system detects this and prints Course exists! to prevent the addition of a duplicate either the input course code is in lowercase or uppercase. No changes are made to the array or counter, keeping existing data intact. Refer to Figure 9 (input in uppercase) and Figure 10 (input in lowercase).

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: CSEB5223
Course exists!
```

Figure 9

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: cseb5223
Course exists!
```

Figure 10

```java

            System.out.print("Course Code: ");
            String code = sc.nextLine();

            if(findCourseIndex(code) != -1) {
                System.out.println("Course exists!"); // Prevent duplicate course entry
                return;
            }
```

Figure 11

Scenario 3: Admin enters non-numeric, decimal number and negative credit hour

If a non-numeric input such as *abc* is entered, Integer.parseInt() throws a NumberFormatException as shown in Figure 13, which is caught to prevent system crashes. The system displays an error message and prompts the user to enter a valid positive integer. For decimal or negative values, the validation loop detects the invalid input and displays: *"Invalid input! Please enter a positive integer for credit hour."* The system continues prompting until a valid value is entered, ensuring only meaningful numeric data is stored based on Figure 12.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: CSEB5113
Course Name: Java Programming
Credit Hour: -8
Invalid input! Please enter a positive integer for credit hour.
Credit Hour: 3.0
Invalid input! Please enter a positive integer for credit hour.
Credit Hour: 3
Summary: Learn Java Programming concepts, OOP and practical coding skills.
Teams Link: https://teams.microsoft.com/l/course/CSEB4113
Course Type (core/elective/university): university
Course added successfully!
```

Figure 12

```java

        System.out.print("Course Name: ");
        String name = sc.nextLine();

        int credit = 0; // store validated credit hour
        while(true) {
            try {
                System.out.print("Credit Hour: ");
                credit = Integer.parseInt(sc.nextLine()); // Convert user input (String) into integer
                if(credit <= 0) throw new NumberFormatException(); // Validate credit must be positive
                break;
            // Error handling to catch non-integer input (example: abc), negative or zero values
            } catch(NumberFormatException e) {
                System.out.println("Invalid input! Please enter a positive integer for credit hour.");
            }
        }
```

Figure 13

Scenario 4: Admin enters invalid (e.g., non-numeric, integer) and valid course type

Figure 14 shows when incorrect course type is entered (example: Abc, 123), the input fails validation. The system prints Invalid course type. Please enter 'core', 'elective' or 'university'. and prompts again. Only word of core, elective and university that contains either lowercase and uppercase letters are accepted as highlighted in Figure 15 and Figure 16, ensuring consistent course classification.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: CSEB5113
Course Name: Java Programming
Credit Hour: 3
Summary: Learn Java programming concepts, OOP and practical coding skills.
Teams Link: https://teams.microsoft.com/l/course/CSEB4113
Course Type (core/elective/university): Abc
Invalid course type. Please enter 'core', 'elective' or 'university'.
Course Type (core/elective/university): 123
Invalid course type. Please enter 'core', 'elective' or 'university'.
Course Type (core/elective/university): CORE
Course added successfully!
```

Figure 14

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: CSEB5113
Course Name: Java Programming
Credit Hour: 3
Summary: Learn Java programming concepts, OOP and practical coding skills.
Teams Link: https://teams.microsoft.com/l/course/CSEB4113
Course Type (core/elective/university): xYz
Invalid course type. Please enter 'core', 'elective' or 'university'.
Course Type (core/elective/university): -9.8
Invalid course type. Please enter 'core', 'elective' or 'university'.
Course Type (core/elective/university): core
Course added successfully!
```

Figure 15

```java
        System.out.print("Summary: ");
        String summary = sc.nextLine();

        System.out.print("Teams Link: ");
        String link = sc.nextLine();

        String type = "";
        while(true) {
            System.out.print("Course Type (core/elective/university): ");
            type = sc.nextLine().toLowerCase();
            // validation of allowed values
            if(type.equals("core") || type.equals("elective") || type.equals("university")) break;
            else System.out.println("Invalid course type. Please enter 'core', 'elective' or 'university'.");
        }

        courses[courseCount++] = new Course(code, name, credit, summary, link, type);
        System.out.println("Course added successfully!");
    }
```

Figure 16

9.3 Search Function Development

With the Search Function, the system allows the user to find a specific course by entering the course code. The program uses a linear search method to check each course in the course list until the matching course code is found. If the course exists, the system will display the complete course attributes such as the course code, course name, credit hour, summary, Teams link, and course type.

To implement this function, the system prompts the user to input the course code. The program then searches through the courses stored in the array. If a match is found, the course information will be displayed. If the course cannot be found in the list, the system will display a message saying “Course not found!” to inform the user that the search was unsuccessful.

The search function allows users to find a course by course code.

1. Implement linear search to find a course by course code
2. Input search criteria
3. Display full course attributes if found
4. Display "Course not found" if not found

Scenario 5: Admin searches for a non-existent course

Figure 17 presents the course code is not found, hence according to Figure 19, findCourseIndex() will return -1, and the system prints Course not found! No other action occurs, clearly informing the admin of an unsuccessful search. Otherwise, if the course code is found, the system will display course information in a readable format as shown in Figure 18.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
A
Enter course code to cache/search: CSEB5143
Course not found!
```

Figure 17

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
a
Enter course code to cache/search: CSEB5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
```

Figure 18

```java

    // Find course index by course code (linear search)
    public int findCourseIndex(String code) {
        for(int i = 0; i < courseCount; i++) {
            if(courses[i].getCourseCode().equalsIgnoreCase(code)) return i;
        }
        return -1;
    }

    // Search for course
    public void searchCourse() {
        System.out.print("Enter course code to search: ");
        int idx = findCourseIndex(sc.nextLine());
        if(idx != -1) courses[idx].displayCourse();
        else System.out.println("Course not found!");
    }

    // Search and return course object
    public Course getCourseByCode(String code) {
        int idx = findCourseIndex(code);
        if(idx != -1) return courses[idx];
        return null;
    }
```

Figure 19

9.4 Edit Function Development

With the Edit Function, the system first performs a search to find the target course that the user wants to edit. The user is required to enter the course code, and the system will search through the course list to locate the matching course. If the course is found, the system will display the complete course attributes. All attributes such as course name, credit hour, summary, Teams link, and course type can be edited except for the course code.

To implement this method, the system prompts the user to enter the course code. Once the course is found, the current course information will be displayed, and the user will be allowed to enter new values to update the course details. After the editing process is completed, the system will display the updated course information to validate the changes. If the course cannot be found, the system will display a message saying, "Course not found!".

Scenario 6: Admin edits course type with invalid input and valid uppercase input

When the admin inputs severe in Figure 20, the system converts the input to lowercase using toLowerCase() and checks it against valid values (core, elective or university). Since 'severe' is not valid, the system prints Invalid type. Type not updated and keeps the original course type unchanged. When the admin enters ELECTIVE in Figure 21, the system again converts it to lowercase (elective) and accepts it as a valid input as shown in Figure 22. The course type is successfully updated, and the system displays the updated course details. This shows that the system handles both invalid inputs and uppercase/lowercase inputs correctly to ensure consistency.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
B
Enter course code to edit: CSEB5223
Editing course: CSEB5223
New Name (Software Construction & Methods):
New Credit (3):
New Summary (Practice coding and build SLMS project.):
New Teams Link (https://teams.microsoft.com/l/course/CSEB5223):
New Type (core): severe
Invalid type. Type not updated.
Course updated successfully!
===================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
===================================
```

Figure 20

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
b
Enter course code to edit: cseb5223
Editing course: CSEB5223
New Name (Software Construction & Methods):
New Credit (3):
New Summary (Practice coding and build SLMS project.):
New Teams Link (https://teams.microsoft.com/l/course/CSEB5223):
New Type (core): ELECTIVE
Course updated successfully!
===================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : elective
===================================
```

Figure 21

```java

    // Edit course
    public void editCourse() {
        System.out.print("Enter course code to edit: ");
        int idx = findCourseIndex(sc.nextLine());

        if(idx != -1) {
            Course c = courses[idx];
            System.out.println("Editing course: " + c.getCourseCode());

            System.out.print("New Name (" + c.getCourseName() + "): ");
            String n = sc.nextLine();
            if(!n.isEmpty()) c.setCourseName(n); // ignore empty input

            System.out.print("New Credit (" + c.getCreditHour() + "): ");
            String cr = sc.nextLine();
            if(!cr.isEmpty()) {
                try {
                    int credit = Integer.parseInt(cr);
                    if(credit > 0) c.setCreditHour(credit);
                    else System.out.println("Credit not updated: must be positive.");
                } catch(NumberFormatException e) {
                    System.out.println("Invalid input for credit.");
                }
            }

            System.out.print("New Summary (" + c.getSummary() + "): ");
            String s = sc.nextLine();
            if(!s.isEmpty()) c.setSummary(s);

            System.out.print("New Teams Link (" + c.getTeamsLink() + "): ");
            String l = sc.nextLine();
            if(!l.isEmpty()) c.setTeamsLink(l);

            System.out.print("New Type (" + c.getCourseType() + "): ");
            String t = sc.nextLine();
            if(!t.isEmpty()) {
                t = t.toLowerCase();
                if(t.equals("core") || t.equals("elective") || t.equals("university")) c.setCourseType(t);
                else System.out.println("Invalid type. Type not updated.");
            }
```

Figure 22

Scenario 7: Admin edits an existing course and a non-existent course

Admin wants to update course CSEB5223 and changes some fields. System finds the course via equalsIgnoreCase(), displays current attributes, and prompts for new values. Empty inputs are ignored, case-normalized inputs are applied and credit hour/type are validated. Updated course info is displayed, confirming successful edit like Figure 25. Please refer Figure 23 for successful editing of output result. In Figure 24, admin enters CSEB5143 which doesn't exist; thus, the system prints Course not found! to inform user and avoid confusion.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
b
Enter course code to edit: CSEB5223
Editing course: CSEB5223
New Name (Software Construction & Methods): Software Construction and Methods
New Credit (3): 4
New Summary (Practice coding and build SLMS project.): Practice coding and build SLMS project using Github and Java Programming.
New Teams Link (https://teams.microsoft.com/l/course/CSEB5223): https://teams.microsoft.com/CSEB5223
New Type (core): UNIVERSITY
Course updated successfully!
================================
Course Code : CSEB5223
Course Name : Software Construction and Methods
Credit Hour : 4
Summary     : Practice coding and build SLMS project using Github and Java Programming.
Teams Link  : https://teams.microsoft.com/CSEB5223
Type        : university
================================
```

Figure 23

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
B
Enter course code to edit: cseb5143
Course not found!
```

Figure 24

```
                System.out.println("Course updated successfully!");
                c.displayCourse();
            } else {
                System.out.println("Course not found!");
            }
        }
```

Figure 25

9.5 Delete Function Development

Users can delete a course safely with confirmation.

With the Delete Function, we needed to first implement the Search Function to find the course. It needed to display the complete course attributes for a successful search before sending a confirmation message to confirm the deletion. However, if the course wasn't able to be found, a message saying "Course not found" would be displayed. Otherwise, the search would display all the courses to validate the deletion of the selected course.

To create this method, we used the searchCourse method to find the selected course that we wanted to delete, by entering the course code. Once found, the system would display the attributes of the selected course, before giving the message to confirm its deletion. If you confirm it, the course is deleted and a message saying "Course deleted successfully!" would be displayed. The system would then display the rest of the courses. If you chose not to confirm it, a message saying "Deletion canceled." will be displayed. If the course is unable to be found, a message saying "Course not found!" will be displayed.

Scenario 8: Admin deletes an existing course

In Figure 26, admin enters course CSEB5223. The system finds it using equalsIgnoreCase() as shown in Figure 30, then displays its details and prompts "Confirm delete? (y/n):". Admin enters capital Y which is accepted by the system due to case-insensitive checking, so the system deletes the course, updates the list, shows "Course deleted successfully!" and display remaining subjects.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
C
Enter course code to delete: CSEB5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): Y
Course deleted successfully!
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
```

Figure 26

Scenario 9: Admin cancels to delete a course

In Figure 27, admin attempts to delete course cseb5223 using small letters which is accepted by the system due to case-insensitive checking . The system finds the course and displays its details, then asks “Confirm delete? (y/n):”. The admin enters n to cancel the operation. The system recognizes the input and stops the deletion process. A message “Deletion canceled.” is displayed and the course remains unchanged in the course list.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: cseb5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): n
Deletion canceled.
```

Figure 27

Scenario 10: Admin attempts to delete a non-existent course

In Figure 28, admin enters course code CSEB5143 to delete. The system searches through the course list using equalsIgnoreCase() but cannot find the course. Since the course does not exist, the system displays the message "Course not found!" to inform the admin and prevent confusion.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: cseb5143
Course not found!
```

Figure 28

Scenario 11: Admin enters invalid confirmation input

Based on Figure 29, admin selects a course to delete and the system displays its details before asking "Confirm delete? (y/n):". Instead of entering y or n, the admin inputs another character such as number 0. Since the input does not match y or Y, the system treats it as a cancellation condition as shown in Figure 30. The deletion process is stopped and canceled, ensuring that the course is not accidentally removed.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: 0
Course not found!
```

Figure 29

```java

    // Delete course and return deleted index
    public int deleteCourse() {
        System.out.print("Enter course code to delete: ");
        int idx = findCourseIndex(sc.nextLine());

        if(idx != -1) {
            courses[idx].displayCourse();
            System.out.print("Confirm delete? (y/n): ");
            if(sc.nextLine().equalsIgnoreCase("y")) {
                // shift elements left
                for(int i = idx; i < courseCount - 1; i++) {
                    courses[i] = courses[i + 1];
                }
                courses[courseCount - 1] = null; // remove last elements
                courseCount--;

                System.out.println("Course deleted successfully!");
                viewAllCourses();
                return idx;
            } else {
                System.out.println("Deletion canceled.");
            }
        } else {
            System.out.println("Course not found!");
        }
        return -1;
    }
```

Figure 30

9.6 Data Display Function Development

The viewAllCourses() method displays all courses in the system.

With the View All Function, we had to make a method when selected, would display all the entered courses and their attributes in a clear and organized manner. This is an important feature that needs to be executed after the Search, Edit or Delete function has completed their instructions. This is necessary to show the result of any changes that occurred after these functions are executed. It also has the option to display itself without the Search, Edit and Delete Functions.

Scenario 12: Display all courses

In Figure 31, admin selects second option to view all courses. The system checks the course list and, since courses are available, it loops through each one and displays their details using displayCourse().

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 2
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
```

Figure 31

Scenario 13: No courses available

Admin selects to view all courses, but the system finds that courseCount == 0 as shown in Figure 33. It then displays the message “No courses.” and returns without showing any course details according to Figure 32.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 2
No courses.
```

Figure 32

```java
    // Display all courses
    public void viewAllCourses() {
        if(courseCount == 0) {
            System.out.println("No courses.");
            return;
        }
        for(int i = 0; i < courseCount; i++) {
            courses[i].displayCourse();
        }
    }

    // Getter for array
    public Course[] getCourses() {
        return courses;
    }

    // Getter for course count
    public int getCourseCount() {
        return courseCount;
    }
}
```

Figure 33

10.0 Student Profile

The Student Profile module of SLMS is responsible for managing student data, including input, search, edit, delete, and display functionalities.

10.1 Class Design

The Student class encapsulates all relevant attributes for a student and provides getter, setter and display methods as shown in Figure 34. It also contains a comment block of the file name which is Student.java, our names, and a brief description of the Student class.

Student.java

Source History

```java
/*
File:
Student.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Represents a student in the SLMS system with attributes, getters, setters, and display functionality.
*/

public class Student {

    // Attributes to store student details
    private String studentID;   // Unique ID
    private String studentName; // Name of student
    private String email;       // Student email
    private String phone;       // Student phone number

    // Constructor: Initializes all course attributes when a Student object is created
    public Student(String studentID, String studentName, String email, String phone) {
        this.studentID = studentID;
        this.studentName = studentName;
        this.email = email;
        this.phone = phone;
    }

    // ------------------------- GETTERS -------------------------
    public String getStudentID() { return studentID; }
    public String getStudentName() { return studentName; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }

    // ------------------------- SETTERS -------------------------
    public void setStudentName(String studentName) { this.studentName = studentName; }
    public void setEmail(String email) { this.email = email; }
    public void setPhone(String phone) { this.phone = phone; }

    // Display student information in readable format
    public void displayStudent() {
        System.out.println("==================================");
        System.out.printf("Student ID   : %s%n", studentID);
        System.out.printf("Student Name : %s%n", studentName);
        System.out.printf("Email        : %s%n", email);
        System.out.printf("Phone        : %s%n", phone);
        System.out.println("==================================");
    }
}
```

Figure 34

Attributes

These private variables store specific data for each student. The access modifier “private” is a practice called encapsulation, which protects the data from being changed directly from outside the class.

Attribute	Type	Description	Example Input
studentID	String	Unique identifier of the student	"BSW01085069"
studentName	String	Full name of the student	"Reyhana Marsya"
email	String	Email address of the student	"rey@student.uniten.edu.my"
phone	String	Contact number of the student	"011-2345678"

Getter and Setter Methods

Since all student attributes are declared private, they cannot be accessed directly from other classes. Therefore, getter and setter methods are used to provide controlled access to the data. Getter methods such as getStudentID(), getStudentName(), getEmail(), and getPhone() allow other parts of the system to retrieve student information safely without modifying it. On the other hand, setter methods including setStudentName(), setEmail(), and setPhone() enable the admin to update student details when necessary. However, the studentID is not provided with a setter method, making it immutable after creation. This ensures data integrity and prevents duplication or accidental modification of the unique student identifier.

Student Display

The displayStudent() method presents student information in a clear, structured console format using System.out.println() and System.out.printf(). It displays attributes such as student ID, name, email and phone number with proper labels and separator lines for better readability. The method improves user understanding and is reusable for functions like searching, viewing all students and listing students in a course.

10.2 Course Input Implementation

The Student Input Implementation allows the admin to add new student information into the Student Learning Management System (SLMS) through console input.

Based on the lab requirements, the system must allow the admin to enter student details and store them in the system. In our implementation, the submit functionality is handled by the addStudent() method in the StudentManager class.

When the admin selects the Add Student option from the menu, the system prompts the user to enter the required student information such as:

- Student ID
- Student Name
- Email
- Phone Number

The system then checks whether the student ID already exists to prevent duplicate entries. If the student ID is already in the system, a message such as “Student exists!” will be displayed and the operation will be cancelled.

If the student ID is unique, a new Student object is created and stored in the students array. The studentCount variable is then increased to keep track of the number of students stored in the system. This implementation ensures that student information is properly stored and managed while maintaining data integrity in the SLMS.

Scenario 1: List of student array is full

In Figure 35, if the student array has reached 100 entries, the system checks studentCount >= students.length from StudentManager class before adding a new student. It prints "Student list full!" and stops the operation, preventing an ArrayIndexOutOfBoundsException as shown in Figure 36. No changes are made, and the system remains stable.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 4
Student list full!
```

Figure 35

```java
/*
File:
StudentManager.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Manages all student-related operations such as add, search, edit, delete and display.
*/

import java.util.Scanner;

public class StudentManager {

    private Scanner sc;
    private Student[] students = new Student[100];
    private int studentCount = 0;

    // Constructor:  Initializes scanner and loads sample data
    public StudentManager(Scanner sc) {
        this.sc = sc;
        loadSampleStudents();
    }

    // Load sample students
    private void loadSampleStudents() {
        students[0] = new Student("BSW01085214", "ARIFAH NUR IMAN BINTI MOHD FADZILAH",
                "BSW01085214@student.uniten.edu.my", "010-1234567");
        students[1] = new Student("BSW01085069", "REYHANA MARSYA BINTI AHMAD SAIFULLIZAN",
                "BSW01085069@student.uniten.edu.my", "011-2345678");
        students[2] = new Student("SW01083593", "DESHINI A/P SELVAKUMAR",
                "SW01083593@student.uniten.edu.my", "012-3456789");
        studentCount = 3;
    }

    // ------------------------- STUDENT FUNCTIONS -------------------------

    // Add student
    public void addStudent() {
        if(studentCount >= students.length) {
            System.out.println("Student list full!");
            return;
        }
```

Figure 36

Scenario 2: Admin try to duplicate existing student

In Figure 37, if the admin enters a student ID that already exists, the method findStudentIndex() returns its index. The system detects this and prints “Student exists!” to prevent duplication as shown in Figure 38. No changes are made to the array or counter, keeping existing data intact.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 4
Student ID: BSW01085069
Student exists!
```

Figure 37

```
            System.out.print("Student ID: ");
            String id = sc.nextLine();

            if(findStudentIndex(id) != -1) {
                System.out.println("Student exists!");
                return;
            }
```

Figure 38

Scenario 3: Admin is successfully input a student

In Figure 39, when valid student details are entered, a new Student object is created and stored in the students array. The studentCount variable is incremented, and the system confirms the operation by displaying “Student added successfully!” as shown in Figure 40.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 4
Student ID: CS24681012
Student Name: Sara
Email: sara@gmail.com
Phone: 012-3456789
Student added successfully!
```

Figure 39

```java
        System.out.print("Student Name: ");
        String name = sc.nextLine();
        System.out.print("Email: ");
        String email = sc.nextLine();
        System.out.print("Phone: ");
        String phone = sc.nextLine();

        students[studentCount++] = new Student(id, name, email, phone);
        System.out.println("Student added successfully!");
    }
```

Figure 40

10.3 Search Function Development

With the Search Function, the admin can search for a specific student by entering their Student ID. Via linear search, the program checks through all the student list database until it finds the matching Student ID. If the Student ID is found, the program will display all information of the student such as their Student ID, name, email address and telephone number.

To implement this function, the admin must input the Student ID of the student they wish to search for. The program runs instructions to look throught the student list in the array, and find the matching Student ID. If a matching Student ID is found, the system shall display the student's information. However, if there is no matching Student ID to be found, the system must display the message saying, "Student not found!" to inform the admin that the search for the student was unsuccessful.

Simply put, the search function allows the admin to search for students, by entering their IDs and finding a matching Student ID in the student list.

1. Implement linear search to find a matching Student ID.
2. Input/Enter Student ID
3. Display student information if a matching Student ID is found
4. Else, display a message saying, "Student not found!" if there is no matching Student ID on the student list.

Scenario 4: Admin searches for a non-existent student

Figure 41 shows the output when the student is not found, and in Figure 43, findStudentIndex() will return –1, and the system will print out "Student not found!". However, if the student is found in the student list, the system will display all the information of the student as shown in Figure 42.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
a
Enter student ID to cache/search: BSW01085393
Student not found!
```

Figure 41

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
A
Enter student ID to cache/search: BSW01085214
==================================
Student ID   : BSW01085214
Student Name : ARIFAH NUR IMAN BINTI MOHD FADZILAH
Email        : BSW01085214@student.uniten.edu.my
Phone        : 010-1234567
==================================
```

Figure 42

```java

    // Find student index by ID
    public int findStudentIndex(String id) {
        for(int i = 0; i < studentCount; i++) {
            if(students[i].getStudentID().equalsIgnoreCase(id)) return i;
        }
        return -1;
    }

    // Search student
    public void searchStudent() {
        System.out.print("Enter student ID to search: ");
        int idx = findStudentIndex(sc.nextLine());
        if(idx != -1) students[idx].displayStudent();
        else System.out.println("Student not found!");
    }

    // Search and return student object
    public Student getStudentById(String id) {
        int idx = findStudentIndex(id);
        if(idx != -1) return students[idx];
        return null;
    }

```

Figure 43

10.4 Edit Function Development

With the Edit Function, the admin must first perform a search for the student, so that they can edit the information of that student. The admin must input the Student ID of the student whose information is to be changed, and the system will search for the matching Student ID in the student list on the system array. If the student is found on the student list, their information is displayed such as name, email address and telephone number. Every attribute can be changed, with the exception of the Student ID.

To implement this function, the admin must enter the Student ID, and the program will perform a search to find the student. When the student is found, their current information shall be displayed, and the admin is able to change the values of each attribute, except the Student ID, to update the student's details. Once the information is updated, the system will display the updated student information to show the changes made by the admin. However, if no matching Student ID is found, the system will display the message saying, “Student not found!”.

Scenario 5: Admin edits information on an existing student and non-existent student

The admin needs to change the details of the student whose Student ID is entered into the system. Through the searchStudent() function, the system runs a program to find the student on the student list and displays the information. There, the admin can input any edits to each attribute, except the Student ID. If the input is empty, no changes are made to the attribute. The updated student info is shown to validate any edits done to the student information, as shown in Figure 44. In Figure 45, when the admin searches for DBSB3272 (a non-existent Student ID), the output will be as what Figure 46 shows a message saying, “Student not found!”.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
b
Enter student ID to edit: BSW01085214
Editing student: BSW01085214
New Name (ARIFAH NUR IMAN BINTI MOHD FADZILAH): GRIGORI RASPUTIN
New Email (BSW01085214@student.uniten.edu.my): russia'sgreatestlovemachine@gmail.com.my
New Phone (010-1234567): 010-6769420
Student updated successfully!
==================================
Student ID   : BSW01085214
Student Name : GRIGORI RASPUTIN
Email        : russia'sgreatestlovemachine@gmail.com.my
Phone        : 010-6769420
==================================
```

Figure 44

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
B
Enter student ID to edit: DBSB3272
Student not found!
```

Figure 45

```java
    // Edit student
    public void editStudent() {
        System.out.print("Enter student ID to edit: ");
        int idx = findStudentIndex(sc.nextLine());

        if(idx != -1) {
            Student s = students[idx];
            System.out.println("Editing student: " + s.getStudentID());

            System.out.print("New Name (" + s.getStudentName() + "): ");
            String n = sc.nextLine();
            if(!n.isEmpty()) s.setStudentName(n);

            System.out.print("New Email (" + s.getEmail() + "): ");
            String e = sc.nextLine();
            if(!e.isEmpty()) s.setEmail(e);

            System.out.print("New Phone (" + s.getPhone() + "): ");
            String p = sc.nextLine();
            if(!p.isEmpty()) s.setPhone(p);

            System.out.println("Student updated successfully!");
            s.displayStudent();
        } else {
            System.out.println("Student not found!");
        }
    }
```

Figure 46

10.5 Delete Function Development

The Delete Function allows the administrator to safely remove a student from the Student Learning Management System (SLMS) with proper confirmation and automatic cleanup of associated course enrollments.

When the admin selects the Delete Student option from the submenu, the system prompts the admin to enter the Student ID of the student to be deleted. The system then searches for the student in the students array using the findStudentIndex() method. If the student is found, the system displays the complete student information to allow the admin to verify the correct student before deletion. The system then asks for confirmation with a prompt such as "Are you sure you want to delete this student? (y/n):".

Scenario 6: Admin deletes an existing student with confirmation.

Figure 47 shows successful student deletion. The system displays student details, requests confirmation, and upon receiving 'Y', deletes the student, shifts remaining array elements left and updates the relationship matrix. It then shows "Student deleted successfully!" and the updated student list.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
c
Enter student ID to delete: BSW01085069
==================================
Student ID   : BSW01085069
Student Name : REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
Email        : BSW01085069@student.uniten.edu.my
Phone        : 011-2345678
==================================
Confirm delete? (y/n): Y
Student deleted successfully!
==================================
Student ID   : BSW01085214
Student Name : ARIFAH NUR IMAN BINTI MOHD FADZILAH
Email        : BSW01085214@student.uniten.edu.my
Phone        : 010-1234567
==================================
==================================
Student ID   : SW01083593
Student Name : DESHINI A/P SELVAKUMAR
Email        : SW01083593@student.uniten.edu.my
Phone        : 012-3456789
==================================
```

Figure 47

Scenario 7: Admin cancels student deletion.

Figure 48 shows the output when the admin chooses to cancel the deletion. After entering the Student ID and viewing the student information, the admin enters 'N' to cancel. The system displays "Deletion canceled!" and returns to the main menu without making any changes to the students array or relationship matrix as shown in Figure 49.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
C
Enter student ID to delete: bsw01085393
==================================
Student ID   : BSW01085393
Student Name : PATVIR SINGH KHOSA
Email        : BSW01085393@student.uniten.edu.my
Phone        : 012-3456789
==================================
Confirm delete? (y/n): N
Deletion canceled.
```

Figure 48

```java

    // Delete student and return deleted index
    public int deleteStudent() {
        System.out.print("Enter student ID to delete: ");
        int idx = findStudentIndex(sc.nextLine());

        if(idx != -1) {
            students[idx].displayStudent();
            System.out.print("Confirm delete? (y/n): ");
            if(sc.nextLine().equalsIgnoreCase("y")) {
                for(int i = idx; i < studentCount - 1; i++) {
                    students[i] = students[i + 1];
                }
                students[studentCount - 1] = null;
                studentCount--;

                System.out.println("Student deleted successfully!");
                viewAllStudents(); // Added to match lab outline
                return idx;
            } else {
                System.out.println("Deletion canceled.");
            }
        } else {
            System.out.println("Student not found!");
        }
        return -1;
    }
```

Figure 49

Scenario 8: Admin attempts to delete a non-existent student.

Figure 50 shows the output when the admin tries to delete a student that does not exist in the system. The findStudentIndex() method returns -1, and the system prints "Student not found!" without performing any deletion operation as shown in Figure 49.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
c
Enter student ID to delete: BSW012345678
Student not found!
```

Figure 50

10.6 Data Display Function Development

The Data Display Function allows the administrator to view all students stored in the Student Learning Management System (SLMS) in a clear and organized manner. This feature provides a comprehensive overview of all student records, enabling the admin to quickly assess the current student population. The display format includes a header with separator lines to improve readability, followed by each student's details including Student ID, Student Name, Email, and Phone Number. Each student's information is separated by a line to clearly distinguish between individual records, making it easy for the admin to scan through the list.

Scenario 9: Admin views all students when multiple students exist in the system.

Figure 51 shows the output when the admin selects the View All Students option and there are existing student records in the system. The system displays each student's information in a structured format with separator lines between records. The student ID, name, email address, and phone number are clearly labeled for each student, allowing the admin to review all enrolled students at once.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 5
==================================
Student ID   : BSW01085214
Student Name : ARIFAH NUR IMAN BINTI MOHD FADZILAH
Email        : BSW01085214@student.uniten.edu.my
Phone        : 010-1234567
==================================
==================================
Student ID   : BSW01085069
Student Name : REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
Email        : BSW01085069@student.uniten.edu.my
Phone        : 011-2345678
==================================
==================================
Student ID   : SW01083593
Student Name : DESHINI A/P SELVAKUMAR
Email        : SW01083593@student.uniten.edu.my
Phone        : 012-3456789
==================================
```

Figure 51

Scenario 10: Admin views all students when no students exist in the system.

Figure 53 shows the output when the admin selects the View All Students option but the students array is empty (studentCount = 0). The system checks the condition and prints "No students." to inform the admin that there are no student records to display as shown in Figure 52. This prevents an empty or confusing output and provides clear feedback to the user.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 5
No students.
```

Figure 52

```java

    // Display all students
    public void viewAllStudents() {
        if(studentCount == 0) {
            System.out.println("No students.");
            return;
        }
        for(int i = 0; i < studentCount; i++) {
            students[i].displayStudent();
        }
    }

    // Getter for array
    public Student[] getStudents() {
        return students;
    }

    // Getter for student count
    public int getStudentCount() {
        return studentCount;
    }
}
```

Figure 53

11.0 Course – Student Relationship and middleware API

The Course–Student Relationship module in the Student Learning Management System (SLMS) is responsible for managing the association between students and courses. This module enables the administrator to assign students to courses, view all courses taken by a student, and list all students enrolled in a specific course.

In this system, the relationship between students and courses is handled by the RelationshipManager class, which acts as a bridge between the StudentManager and CourseManager. This ensures that both student and course data can interact without directly depending on each other, thus maintaining low coupling and better system modularity.

Additionally, a middleware component is implemented using the SearchCacheManager class. This component acts as a simple API layer that stores recent search inputs and provides suggestion functionality. It improves user experience by assisting input and reducing repetitive typing.

Overall, this module demonstrates the use of structured data handling, modular design, and basic middleware integration in a console-based Java application.

11.1 Data Structure Design

Course Attributes

The Course class is used to store course-related information such as course code, name and type. These attributes represent the essential details of a course. The courseType allows classification such as core, elective or university.

```java
public class Course {

    // Attributes to store course information
    private String courseCode; // Unique course code
    private String courseName; // Name of the course
    private int creditHour;    // Number of credit hours
    private String summary;    // Brief description of the course
    private String teamsLink;  // Microsoft Teams link
    private String courseType; // Type: core, elective, university
```

Figure 54

Student Attributes

The Student class stores student-related information. Each student is uniquely identified using studentID, while other attributes store personal and contact information. The system uses arrays to store multiple student and course objects.

```
public class Student {

    // Attributes to store student details
    private String studentID;   // Unique ID
    private String studentName; // Name of student
    private String email;       // Student email
    private String phone;       // Student phone number

```

Figure 55

Course Array Initialization

The Course Array is used to store multiple course objects in a similar way. It maintains up to 100 course records, with courseCount tracking the number of active entries, ensuring safe storage management and preventing data overflow.

```
    private Scanner sc;
    private Course[] courses = new Course[100]; // Fixed-size storage
    private int courseCount = 0; // Tracks number of stored courses
```

Figure 56

Student Array Initialization

The Student Array is used to store multiple student objects in a structured collection. It allows the system to manage up to 100 student records efficiently, while studentCount keeps track of how many students are currently stored to prevent overflow and control data access.

```
    private Scanner sc;
    private Student[] students = new Student[100];
    private int studentCount = 0;
```

Figure 57

Student–Course Relationship Attributes

These private variables store and manage the relationship data.

Attribute	Type	Description
sc	Scanner	Used to capture user input
courseManager	CourseManager	Access course data
studentManager	StudentManager	Access student data
relationship	boolean[][]	Stores course-student relationships

Student–Course Relationship Array Initialization

The relationship between students and courses is managed using a two-dimensional boolean array inside the RelationshipManager class. In this structure, each row represents a student and each column represents a course. If the value at relationship[studentIndex][courseIndex] is true, it indicates that the student is enrolled in that course. If the value is false, it means no relationship exists between the student and the course. This design allows efficient tracking of many-to-many relationships between students and courses.

RelationshipManager.java

Source History

```java
/*
File:
RelationshipManager.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Manages relationships between students and courses using a 2D boolean array.
Handles assigning, finding, and listing course-student relationships.
*/

import java.util.Scanner;

public class RelationshipManager {

    private Scanner sc;
    private CourseManager courseManager;
    private StudentManager studentManager;

    // 2D relationship array: relationship[studentIndex][courseIndex]
    private boolean[][] relationship = new boolean[100][100];

    // Constructor
    public RelationshipManager(Scanner sc, CourseManager courseManager, StudentManager studentManager) {
        this.sc = sc;
        this.courseManager = courseManager;
        this.studentManager = studentManager;
        loadSampleRelationships();
    }

    // Load sample assignments
    private void loadSampleRelationships() {
        relationship[0][0] = true; // Student 0 -> Course 0
        relationship[1][1] = true; // Student 1 -> Course 1
        relationship[2][2] = true; // Student 2 -> Course 2
    }

```

Figure 58

11.2 Relationship Management Functions

AssignStudentToCourse() Function

The main function for managing relationships is assignStudentToCourse(). This function allows the user to assign a student to a selected course by entering the student ID and course code. The system first searches for the student index and course index using the respective manager classes. If either the student or course is not found, an error message is displayed and the process is stopped. If both exist, the system checks whether the relationship already exists. If the student is already enrolled in the course, the system prevents duplication and displays a message. Otherwise, the relationship is stored in the 2D boolean array and marked as true.

```java

    // ------------------------- RELATIONSHIP FUNCTIONS -------------------------

    // Assign student to course
    public void assignStudentToCourse() {
        System.out.print("Student ID: ");
        String sid = sc.nextLine();
        System.out.print("Course Code: ");
        String code = sc.nextLine();

        int sIdx = studentManager.findStudentIndex(sid);
        int cIdx = courseManager.findCourseIndex(code);

        if(sIdx == -1) {
            System.out.println("Student not found!");
            return;
        }

        if(cIdx == -1) {
            System.out.println("Course not found!");
            return;
        }

        if(relationship[sIdx][cIdx]) {
            System.out.println("Already assigned! Student is already enrolled in this course.");
            return;
        }

        relationship[sIdx][cIdx] = true;
        System.out.println("Assignment successful!");
    }
```

Figure 59

addCourse() Function

The addCourse() function does not directly create a course. Instead, it reuses the assignStudentToCourse() method. This design demonstrates code reuse, where the same logic is used to assign a student to a course. It simplifies the system by avoiding duplicate implementation.

```java

    // addCourse() to student relationship
    public void addCourse() {
        assignStudentToCourse();
    }
```

Figure 60

addStudent() Function

Similarly, the addStudent() function also calls assignStudentToCourse(). This allows the system to reuse the same assignment logic for student-course relationships. This approach improves modularity and ensures consistent behavior across different functions.

```java

    // addStudent() to course relationship
    public void addStudent() {
        assignStudentToCourse();
    }

```

Figure 61

findCourse() and listCourses()

The findCourse() function is used to retrieve all courses assigned to a specific student. It internally calls the listCourses() method. In listCourses(), the system first takes the student ID as input and finds the corresponding student index. It then loops through all courses and checks the relationship array. If a value is true, the course is displayed as part of the student's enrolled courses. If no courses are found, the system displays a message indicating that the student has no assigned courses.

```java
    // Find student's course(s)
    public void findCourse() {
        listCourses();
    }

    // List all courses of a student
    public void listCourses() {
        System.out.print("Enter Student ID: ");
        String sid = sc.nextLine();

        int sIdx = studentManager.findStudentIndex(sid);

        if(sIdx == -1) {
            System.out.println("Student not found!");
            return;
        }

        boolean found = false;
        System.out.println("Courses enrolled:");

        for(int c = 0; c < courseManager.getCourseCount(); c++) {
            if(relationship[sIdx][c]) {
                courseManager.getCourses()[c].displayCourse();
                found = true;
            }
        }

        if(!found) {
            System.out.println("This student has no assigned course.");
        }
    }

```

Figure 62

findStudent() and listStudents()

The findStudent() function retrieves all students assigned to a specific course by calling the listStudents() method. In this method, the system takes a course code as input and finds its index. It then loops through all students and checks the relationship matrix. If the value is true, the student is displayed as enrolled in that course. If no students are found, the system displays a message stating that no students are assigned to the course.

```java
    // Find student(s) in a course
    public void findStudent() {
        listStudents();
    }

    // List all students of a course
    public void listStudents() {
        System.out.print("Enter Course Code: ");
        String code = sc.nextLine();

        int cIdx = courseManager.findCourseIndex(code);

        if(cIdx == -1) {
            System.out.println("Course not found!");
            return;
        }

        boolean found = false;
        System.out.println("Students enrolled:");

        for(int s = 0; s < studentManager.getStudentCount(); s++) {
            if(relationship[s][cIdx]) {
                studentManager.getStudents()[s].displayStudent();
                found = true;
            }
        }

        if(!found) {
            System.out.println("This course has no assigned student.");
        }
    }

    // Remove course relationships after course deletion
    public void removeCourseRelationship(int deletedCourseIndex) {
        if(deletedCourseIndex == -1) return;

        for(int s = 0; s < studentManager.getStudentCount(); s++) {
            for(int c = deletedCourseIndex; c < courseManager.getCourseCount(); c++) {
                relationship[s][c] = relationship[s][c + 1]; //Prevent overflow
            }
            relationship[s][courseManager.getCourseCount()] = false;
        }
    }
```

Figure 63

11.3 Error Handling and Edge Cases

The Relationship Manager module is designed to ensure system stability through structured validation checks and controlled execution flow. Before any student is assigned to a course, the system performs index-based validation to confirm that both the student and course exist in the system. If either entity cannot be found, the operation is immediately terminated to prevent invalid or corrupted data processing.

The system uses a 2D boolean relationship matrix to manage student–course mappings. Each assignment is validated against the matrix to prevent duplicate entries, ensuring that an existing relationship cannot be reassigned. This maintains data integrity across all operations.

For retrieval operations, the system includes handling for empty relationship cases. When no data is available (example: a student is not enrolled in any course or a course has no assigned students), the system displays a clear and user-friendly message instead of returning an empty output.

In addition, when a student or course is deleted, the system automatically updates the relationship matrix using controlled shifting operations to remove all associated references. This ensures that no mismatched data remains in the system.

Overall, these error-handling mechanisms ensure robust system behavior, consistent data integrity, and clear user feedback across all operations.

Scenario 1: Admin successfully assigns a student to a course using assignStudentToCourse()

Figure 64 shows the output when the admin selects the Assign Student to Course option from the menu. The system prompts the admin to enter a Student ID and a Course Code, for example 'BSW01085214' and 'CSEB4113'. The system then executes the assignStudentToCourse() function, which validates both inputs by calling findStudentIndex() from StudentManager and findCourseIndex() from CourseManager. After both indices are confirmed to be valid, the system updates the relationship matrix by setting relationship[sIdx][cIdx] to true. The system then displays the message "Assignment successful!".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 7
Student ID: BSW01085214
Course Code: CSEB4113
Assignment successful!
```

Figure 64

Scenario 2: Assignment fails due to invalid student ID

Figure 65 shows the output when the admin selects the Assign Student to Course option but enters an invalid student ID such as 'WOW123456'. The system executes the assignStudentToCourse() function and attempts to locate the student using findStudentIndex(). Since the student does not exist, the function returns -1 and the system displays the message "Student not found!". No changes are made to the relationship matrix.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 7
Student ID: WOW123456
Course Code: CSEB5223
Student not found!
```

Figure 65

Scenario 3: Assignment fails due to invalid course code

Figure 66 shows the output when the admin enters a valid student ID but an invalid course code such as 'meow12345'. The system executes the assignStudentToCourse() function and calls findCourseIndex(). Since the course does not exist, the function returns -1 and the system displays the message "Course not found!". No assignment is performed in the relationship matrix.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 7
Student ID: BSW01085214
Course Code: meow12345
Course not found!
```

Figure 66

Scenario 4: Admin attempts duplicate assignment

Figure 67 shows the output when the admin tries to assign a student to a course they are already enrolled in using the 7th option. The system executes the assignStudentToCourse() function and checks the relationship matrix. Since the value relationship[sIdx][cIdx] is already true, the system detects the duplicate and displays the message "Already assigned! Student is already enrolled in this course". No duplicate entry is created.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 7
Student ID: bsw01085214
Course Code: CSEB5223
Already assigned! Student is already enrolled in this course.
```

Figure 67

Scenario 5: Admin views all courses of a student using listCourses()

Figure 68 shows the output when the admin selects the List Courses of Student option. The system executes the listCourses() function and prompts the admin to enter a Student ID. After validating the student index using StudentManager, the system loops through all courses in the relationship matrix and checks which ones are assigned to the student. All matched courses are displayed using the displayCourse() method under the "Courses enrolled".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 8
Enter Student ID: BSW01085214
Courses enrolled:
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
```

Figure 68

Scenario 6: Student has no assigned courses

Figure 69 shows the output when the admin selects the 8th option, but the selected student has no assigned courses. The system executes the listCourses() function, but no true values are found in the relationship matrix row for that student. As a result, the system displays the message "This student has no assigned course.".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 8
Enter Student ID: BSW1234567
Courses enrolled:
This student has no assigned course.
```

Figure 69

Scenario 7: Admin views all students in a course using listStudents()

Figure 70 shows the output when the admin selects the List Students of Course option. The system executes the listStudents() function and prompts the admin to enter a Course Code. After validating the course index using CourseManager, the system scans through all students in the relationship matrix column. All students assigned to the course are displayed using the displayStudent() method under the "Students enrolled" section.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 9
Enter Course Code: CCSB5113
Students enrolled:
==================================
Student ID   : SW01083593
Student Name : DESHINI A/P SELVAKUMAR
Email        : SW01083593@student.uniten.edu.my
Phone        : 012-3456789
==================================
```

Figure 70

Scenario 8: Course has no assigned students

Figure 71 shows the output when the admin selects the 9th option but no students are assigned to the selected course. The system executes the listStudents() function and finds no true values in the relationship matrix column. The system then displays the message "This course has no assigned student.".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 9
Enter Course Code: CSEB123456
Students enrolled:
This course has no assigned student.
```

Figure 71

Scenario 9: Admin deletes a course and relationship matrix is updated

Figure 72 shows the output after the admin deletes course from the system using CourseManager. The system automatically calls the removeCourseRelationship() function, which shifts all relationship matrix columns to remove references to the deleted course. The last column is cleared to prevent data overflow. The updated relationships are shown, where all student-course links remain consistent after deletion. After this operation, the admin selects the 1st option to view all courses and also selects the 8th option to list courses of student to prove and verify the updated records.

Scenario 10: Admin deletes a student and relationship matrix is updated

Figure 73 shows the output after the admin deletes a student using StudentManager. The system executes the removeStudentRelationship() function, which shifts all rows in the relationship matrix upward to remove the deleted student's data. The last row is then cleared to avoid leftover or invalid entries. As a result, the updated matrix is displayed, ensuring all remaining student–course relationships remain valid and consistent. After the deletion process, the admin views the updated system state through View All Students option, where only the remaining students are listed. Finally, the admin selects the 9th option whis is List Students of Course for course CSEB5223, and the system confirms that no students are currently assigned to the course, reflecting the updated relationships accurately.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
C
Enter course code to delete: CSEB5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): Y
Course deleted successfully!
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 2
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 8
Enter Student ID: BSW01085214
Courses enrolled:
This student has no assigned course.
```

Figure 72

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 6
a. Search Student
b. Edit Student
c. Delete Student
c
Enter student ID to delete: bsw01085214
==================================
Student ID   : BSW01085214
Student Name : ARIFAH NUR IMAN BINTI MOHD FADZILAH
Email        : BSW01085214@student.uniten.edu.my
Phone        : 010-1234567
==================================
Confirm delete? (y/n): y
Student deleted successfully!
==================================
Student ID   : BSW01085069
Student Name : REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
Email        : BSW01085069@student.uniten.edu.my
Phone        : 011-2345678
==================================
==================================
Student ID   : SW01083593
Student Name : DESHINI A/P SELVAKUMAR
Email        : SW01083593@student.uniten.edu.my
Phone        : 012-3456789
==================================

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 5
==================================
Student ID   : BSW01085069
Student Name : REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
Email        : BSW01085069@student.uniten.edu.my
Phone        : 011-2345678
==================================
==================================
Student ID   : SW01083593
Student Name : DESHINI A/P SELVAKUMAR
Email        : SW01083593@student.uniten.edu.my
Phone        : 012-3456789
==================================

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 9
Enter Course Code: CSEB5223
Students enrolled:
This course has no assigned student.
```

Figure 73

11.4 API Implementation

The SLMS system implements a simple middleware API using the SearchCacheManager class to improve user experience through caching and auto-suggestions.

A. Caching of Search and Input Data in textfields or forms

The system stores recently searched course codes and student IDs in arrays within the SearchCacheManager. When a user performs a search, the input is cached using methods like cacheCourseSearch(String courseCode) and cacheStudentSearch(String studentId). These methods save the values into arrays (recentCourseSearches and recentStudentSearches) and update their counters. This approach allows the system to retain frequently searched data without relying on an external database, acting as a lightweight API layer that temporarily manages data during runtime.

SearchCacheManager.java

Source History

```java
/*
File:
SearchCacheManager.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Acts as a simple middleware/API cache layer for storing recent searches
and providing basic auto-suggestions for input fields.
*/

public class SearchCacheManager {

    private String[] recentCourseSearches = new String[20];
    private String[] recentStudentSearches = new String[20];
    private int courseSearchCount = 0;
    private int studentSearchCount = 0;

    // Cache recent course search
    public void cacheCourseSearch(String courseCode) {
        if(courseSearchCount < recentCourseSearches.length) {
            recentCourseSearches[courseSearchCount++] = courseCode;
        }
    }

    // Cache recent student search
    public void cacheStudentSearch(String studentId) {
        if(studentSearchCount < recentStudentSearches.length) {
            recentStudentSearches[studentSearchCount++] = studentId;
        }
    }

```

Figure 74

B. Auto-suggestion Based on Cached Data in input fields

The system provides auto-suggestions by comparing user input with cached values. When partial input is entered, it calls suggestCourses(String input) or suggestStudents(String input). These methods loop through the stored data and display matches using case-insensitive partial string matching with contains(). For example, entering "CSE" may suggest course codes like CSEB5223, while "BSW" may return matching student IDs. This feature improves usability by reducing typing effort and helping users quickly find relevant results.

```java
    // Show cached course suggestions
    public void suggestCourses(String input) {
        System.out.println("Suggested Course Codes:");
        boolean found = false;

        for(int i = 0; i < courseSearchCount; i++) {
            if(recentCourseSearches[i].toLowerCase().contains(input.toLowerCase())) {
                System.out.println("- " + recentCourseSearches[i]);
                found = true;
            }
        }

        if(!found) {
            System.out.println("No suggestions found.");
        }
    }

    // Show cached student suggestions
    public void suggestStudents(String input) {
        System.out.println("Suggested Student IDs:");
        boolean found = false;

        for(int i = 0; i < studentSearchCount; i++) {
            if(recentStudentSearches[i].toLowerCase().contains(input.toLowerCase())) {
                System.out.println("- " + recentStudentSearches[i]);
                found = true;
            }
        }

        if(!found) {
            System.out.println("No suggestions found.");
        }
    }
}
```

Figure 75

Scenario 1: Admin caches a course search using cacheCourseSearch()

Figure 76 shows the output when the admin selects the Course Search Suggestions option from the system menu and enters a course code such as 'PKIB113'. The system then calls the function cacheCourseSearch(String courseCode), which stores the entered course code into the recentCourseSearches array. The courseSearchCount variable is incremented to keep track of the number of stored searches. The system does not display a message for this action, but the data is successfully stored in the cache.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 10
Enter partial course code: PKIB113
Suggested Course Codes:
No suggestions found.
```

Figure 76

Scenario 2: Admin caches a student search using cacheStudentSearch()

Figure 77 shows the output when the admin selects the Student Search Suggestions option and enters a student ID such as 'CS01023456'. The system executes the function cacheStudentSearch(String studentId), which stores the student ID into the recentStudentSearches array. The studentSearchCount variable is then increased accordingly. The system successfully saves the search history without displaying output and the updated cache.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 11
Enter partial student ID: CS01023456
Suggested Student IDs:
No suggestions found.
```

Figure 77

Scenario 3: Admin receives course suggestions using suggestCourses()

Figure 78 shows the output when the admin selects the 10th option and types a partial input such as 'cSeB'. The system calls the function suggestCourses(String input), which scans through the recentCourseSearches array and compares each stored value with the input in a case-insensitive manner. When matches are found, the system displays the matching course codes under the "Suggested Course Codes".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 10
Enter partial course code: cSeB
Suggested Course Codes:
- cseb5223
- CSEb 5113
- Cseb5112
- CseB5113
- csEB4113
```

Figure 78

Scenario 4: No course suggestions are found

Figure 79 shows the output when the admin selects the Course Search Suggestions option but enters an input such as 'WoW' that does not match any cached values. The system executes the suggestCourses(String input) function and iterates through all stored course searches. Since no matches are found, the system sets the found variable to false and displays the message "No suggestions found.".

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 10
Enter partial course code: WoW
Suggested Course Codes:
No suggestions found.
```

Figure 79

Scenario 5: Admin receives student suggestions using suggestStudents()

Figure 80 shows the output when the admin selects the 11th option and enters a partial input such as ' sW'. The system executes the suggestStudents(String input) function, which searches through the recentStudentSearches array and checks for matching values. When matches are found, the system displays the relevant student IDs under the "Suggested Student IDs" section.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 11
Enter partial student ID: sW
Suggested Student IDs:
- bsW01085214
- bsw01085069
- SW01083593
```

Figure 80

12.0 Analysis of software construction best practices and challenges

Now that we have made this program, let us go over on what practices of software construction were used, and what types of challenges were faced during the development cycle. With this, we can report on what are the best practices to use for specific projects and how to overcome any challenge that happens during the development cycle.

12.1 Lab 3 & Lab 4

Labs 3 and 4 were focused on developing the course module of the program. Functions such as Search, Edit, Delete, and Data Display were implemented as well as Course Input Implementation, which allowed the administrators to add and change course information and store it in the system. There are attributes such as courseCode, courseName, creditHour, summary, teamLink and courseType. The inputs included in Course Input Implementation to add a course are Course Code, Course Name, Credit Hour, Course Summary, MS Team Link, and Course Type.

With Course Input Implementation, we needed to validate the input given by an administrator, so it would store into the system without any errors. By introducing this function, we can prevent duplicate entries, keep track of the number of courses in the system, and stores information in an organized and maintainable state.

Since we needed a way to store information, we created a Data Structure using an array to store course information. When adding a new course, the administrator must enter an entirely new course code, in order to create a new Course Object which is then stored into the system. Because we used an array, we are able to store multiple courses without any out-of-bound errors occurring. Not only that, but we also reused code from other functions such as Search Course in multiple functions, and Display Course after every function is performed.

One major challenge was handling requirement changes, meaning we had to code new functions like Edit, Delete, and View All Course Functions as well as duplication prevention into the program. We managed to implement those functions using inheritance (where we reused code from previously existing functions) and encapsulation (where we combine methods and variables into classes) to allow implementation of new codes without affecting any of the previous codes.

12.2 Lab 5 & Lab 6

During Lab 5 and 6, we developed the Student Profile Module. In that module, we created a new Class (Student) and added attributes such as studentID, studentName, email, and phone. With that, we were able to implement new functions like Search, Add, Edit, Delete Student, etc. which are used as inputs for the system. With the input validation programmed into the system, any input given by an administrator on student information is stored into the system. Because of the implementation, we are able to prevent any duplications and manage the student information in the system.

When an administrator adds a new student, they must first enter a unique student ID to create a new Student in the system, before filling out the rest of their details. By making a Data Structure like an array, we can store multiple students into the system, similar to how we stored course data on Lab 3 and 4. Not only that, but we also reused code from other functions such as Search Student in multiple functions, and Display Students after every function is performed.

A major hurdle we face when developing this section of the program was implementing new features (requirement handling). We had to code new codes for functions like Add, Search, Edit Student, etc. We managed to overcome this challenge by using inheritance (reusing code from previous codes to help build our new functions), and encapsulation (combining variables and methods into classes) to implement these new functions into the system.

12.3 Lab 7 & Lab 8

For this current lab (7 & 8), we focused on Student Relationships and middleware API, where we needed to implement student-course relationships. We first had to design a Data Structure diagram to visualize the relationships between students and their assigned courses. We did this by making an array that stored Course and Student objects.

Afterwards, we then had to implement Relationship Management Functions such as Add Course, Add Student, List Courses, List Students, Find Course and Find Student. With this, we also needed to minimize errors and edge casing by implementing checks and sending messages to the user to notify them of any errors that have occurred. Lastly, we had to implement an API into the system to cache searches and inputs.

One problem we faced when developing this section of the program was once again, implementing the new features (requirement handling). By using inheritance (reusing code from previous functions to make new functions0 and encapsulation (combining methods and variables into classes), we were able to overcome this situation and implement these new requirements into the system without affecting the previous functions.

13.0 System Testing

System testing is performed on the complete Student Learning Management System (SLMS) as a fully integrated application. This testing ensures that all modules, including CourseManager, StudentManager, RelationshipManager and SearchCacheManager, functions correctly together from the administrator's perspective. The system is executed using a menu-driven interface consisting of options 1 to 11 and an exit option 0.

The system interface presented to the admin is shown below in Figure 81:

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice:
```

Figure 81

The following scenarios simulate real user interactions from the beginning of program execution until termination, including both valid and error-prone inputs to observe system behavior.

Scenario 1: Full Course Management Flow

In this scenario, the administrator manages a course throughout its lifecycle. The process begins by adding a new course, followed by editing its details and finally deleting the course. This scenario verifies input validation, data consistency and proper array handling within the system.

In the first step, the administrator selects menu option 1 to create a new course in Figure . The user enters the course code as 'PKIB113' and the course name as 'Creative Thinking'. When entering the credit hour, the system first receives an invalid input "meOw", followed by another invalid input "-3". Both inputs are rejected by the system. Finally, when the user enters the value "3", it is accepted as valid input. The course type is then entered as "", and the system successfully adds the course. This behavior is controlled by a try-catch block that ensures only positive integers are accepted for credit hours. The system displays error messages when invalid inputs are entered and continues prompting until correct data is provided. This demonstrates effective defensive programming as the system prevents crashes and ensures correct data entry.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 1
Course Code: PKIB113
Course Name: Creative Thinking
Credit Hour: meOw
Invalid input! Please enter a positive integer for credit hour.
Credit Hour: -3
Invalid input! Please enter a positive integer for credit hour.
Credit Hour: 3
Summary: This course encourages you to explore unconventional ways of thinking.
Teams Link: https://teams.microsoft.com/l/course/PKIB113
Course Type (core/elective/university): elective
Course added successfully!
```

Figure 82

In the second step, the administrator edits the previously added course by selecting menu option 3 followed by option b in Figure 83. During the update process, the user attempts to enter an invalid credit hour of “-5”, which is rejected because it is not a positive value. The user then enters “abc”, which triggers a NumberFormatException and is also rejected. Finally, the user enters a valid credit hour of “4”, which is accepted and successfully updates the course. The system ensures that invalid updates do not overwrite existing valid data, thereby maintaining data integrity. Appropriate error messages are displayed for each invalid input.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
b
Enter course code to edit: PKIB113
Editing course: PKIB113
New Name (Creative Thinking):
New Credit (3): -5
Credit not updated: must be positive.
New Credit (3): abc
Invalid input for credit.
New Credit (3): 4
New Summary (This course encourages you to explore unconventional ways of thinking.):
New Teams Link (https://teams.microsoft.com/l/course/PKIB113):
New Type (elective):
Course updated successfully!
==================================
Course Code : PKIB113
Course Name : Creative Thinking
Credit Hour : 4
Summary     : This course encourages you to explore unconventional ways of thinking.
Teams Link  : https://teams.microsoft.com/l/course/PKIB113
Type        : elective
==================================
```

Figure 83

In the final step, the administrator deletes the course ‘cseb5223’ using menu option 3 followed by option c in Figure 84. After confirmation, the system removes the course from the array and shifts all subsequent elements to fill the gap. This is done to ensure that there are no empty spaces in the array, which could lead to null pointer issues. The system then updates the course count accordingly and confirms that the course has been deleted successfully. This demonstrates correct array management and proper handling of dynamic data within a fixed-size structure.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: cseb5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): Y
Course deleted successfully!
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
==================================
Course Code : PKIB113
Course Name : Creative Thinking
Credit Hour : 4
Summary     : This course encourages you to explore unconventional ways of thinking.
Teams Link  : https://teams.microsoft.com/l/course/PKIB113
Type        : elective
==================================
```

Figure 84

Scenario 2: Student Registration and Course Assignment Flow

This scenario focuses on the integration between StudentManager, CourseManager and RelationshipManager. It tests the process of adding a student, assigning the student to a course and verifying the relationship between them.

In the first step, the administrator selects menu option 4 to add a new student. The user enters the student ID as 'SW01056789', the name as 'Barack Hussein Obama', the email as 'SW01056789@student.uniten.edu.my' and the phone number as '010-98765432'. The system stores this information successfully and confirms that the student has been added. It also ensures that duplicate student IDs are not allowed, maintaining data uniqueness.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 4
Student ID: SW01056789
Student Name:  Barack Hussein Obama
Email: SW01056789@student.uniten.edu.my
Phone: 010-98765432
Student added successfully!
```

Figure 85

In the second step, the administrator assigns the student to a course using menu option 7 in Figure 86. The system requires both the student ID and course code. If either the student or the course does not exist, the system displays an error message and stops the operation. In this case, since both ‘SW01056789’ and ‘pkib113’ exist, the assignment is successful. The system records this relationship in a two-dimensional boolean array, ensuring that the student is linked to the selected course.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 7
Student ID: SW01056789
Course Code: pkib113
Assignment successful!
```

Figure 86

In the third step, the administrator selects menu option 8 to list all courses assigned to the student ID ‘Sw01056789’. The system retrieves and displays the course ‘PKIB113’, confirming that the relationship has been stored correctly as shown in Figure 87. This demonstrates that the relationship mapping between students and courses is functioning as expected.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 8
Enter Student ID: Sw01056789
Courses enrolled:
==================================
Course Code : PKIB113
Course Name : Creative Thinking
Credit Hour : 4
Summary     : This course encourages you to explore unconventional ways of thinking.
Teams Link  : https://teams.microsoft.com/l/course/PKIB113
Type        : elective
==================================
```

Figure 87

Scenario 3: Search, Cache and System Exit Flow

This scenario demonstrates system wide navigation, including search operations, caching functionality, suggestion generation and safe system termination.

In the first step, the administrator searches for a course using menu option 3 followed by option a in Figure 88. The course code 'pKIB113' is entered, and the system successfully retrieves and displays the course details. At the same time, the search input is stored in the cache through the SearchCacheManager, allowing it to be reused for suggestions later.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
a
Enter course code to cache/search: pKIB113
==================================
Course Code : PKIB113
Course Name : Creative Thinking
Credit Hour : 4
Summary     : This course encourages you to explore unconventional ways of thinking.
Teams Link  : https://teams.microsoft.com/l/course/PKIB113
Type        : elective
==================================
```

Figure 88

In the second step, the administrator selects menu option 10 to view course suggestions. When the partial input 'PK' is entered in Figure 89, the system searches the cached data and displays matching course codes such as 'pKIB113'. This feature improves user experience by reducing the need to type full course codes and demonstrates the usefulness of caching mechanisms.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 10
Enter partial course code: PK
Suggested Course Codes:
- pKIB113
```

Figure 89

In the final step, the administrator selects menu option 0 to exit the system. The system displays a message indicating that it is exiting and terminates the program safely. Resources such as the Scanner are properly closed, ensuring a clean and controlled shutdown of the application as shown in Figure 90.

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
10. Course Search Suggestions
11. Student Search Suggestions
0. Exit
Enter choice: 0
Exiting system...
BUILD SUCCESSFUL (total time: 36 minutes 16 seconds)
```

Figure 90

System Testing Analysis

The system demonstrates correctness as all functions behave as expected under both valid and invalid inputs. The outputs produced by the system align with the intended design and functionality of each module. Defensive programming techniques are clearly implemented throughout the system, including the use of try-catch blocks for handling invalid numeric inputs, validation checks for ensuring positive values, duplicate checking for unique entries and null checks during search operations. These measures prevent runtime errors and ensure system stability.

Integration testing confirms that all modules interact effectively with each other. The CourseManager and StudentManager are successfully connected through the RelationshipManager, while the SearchCacheManager enhances search functionality by storing and suggesting previous inputs. Data consistency is maintained across all operations, even when records are added, edited or deleted.

Despite the successful implementation, several challenges were encountered during development. Managing array shifting after deletion required careful handling to avoid data loss and ensure correct indexing. Maintaining relationship consistency between students and courses after deletion operations was also complex. Additionally, ensuring that all user inputs were properly validated without interrupting the overall system flow required careful design and testing.

14.0 Team Contributions

Throughout the project, all team members contributed across different stages of development, from initial planning to final submission. Tasks were distributed to ensure efficiency, steady progress and balanced responsibility among members.

At the initial stage, the team collaboratively defined project objectives, interpreted requirements and developed the overall structure of the report. Responsibilities were assigned based on individual strengths and areas of competence.

During the development phase, members worked on various components including research, data collection, analysis and content development. Regular discussions were held through Telegram call and message to maintain consistency and alignment with project objectives, while issues encountered were resolved through certain effective communication and mutual support.

In the later stages, members focused on refining their respective sections, improving content quality and ensuring accuracy. Peer review and collaborative feedback were used to enhance coherence across the report. The final stage involved compiling all sections, formatting, and proofreading to ensure the report met academic standards.

Although contributions varied according to individual circumstances, one member showed strong early progress but became less consistent later, another faced technical difficulties which caused some delays, while another maintained consistent performance and provided regular progress updates that supported team coordination.

Overall, despite differences in pace and challenges faced, all members contributed meaningfully. The project was successfully completed through teamwork, coordination and collective effort.

15.0 References

- Moodle Development. (n.d.). MoodleDev: Moodle developer documentation. https://moodledev.io/
- Moodle. (n.d.). Moodle: The open-source learning platform (Source code). GitHub. https://github.com/moodle/moodle
- SourceMaking. (n.d.). SourceMaking: Software design patterns, refactoring, and UML examples. https://sourcemaking.com/
- GitByBit. (n.d.). GitByBit: Git tutorials, tips, and guides. https://gitbybit.com/
- draw.io. (n.d.). *draw.io: Online diagram software.* https://www.drawio.com/